

Recursion



Understanding recursive functions in C programming



What is Recursion?

Recursion is a process by which function calls itself repeatedly, until some specified condition has been satisfied. The process is used for repetitive computation in which each action is stated in terms of previous result.

✓ Conditions for Recursive Solutions

- Problem must be written in recursive form
- Problem statement must include a stopping condition

Recursive Function Definition

Recursive function is a function that contains a call to itself. C supports creating recursive function with ease and efficient.



Exit Condition

Recursive function must have at least one exit condition that can be satisfied



Stack Overflow

Without exit condition, the recursive function will call itself repeatedly until the runtime stack overflows



Problem Solving

Recursive function allows you to divide your complex problem into identical single simple cases

This is also a well-known computer programming technique

Example 1: Factorial Function



Factorial Calculation Using Recursion

```
#include<stdio.h>
long unsigned int factorial( long unsigned int number)
{
    if(number <= 1)
        return 1;
    else
        return number * factorial(number - 1);
}
void main()
{
    long unsigned int N,F;
    clrscr( );
    printf("Enter any Number for Calculate Factorial : ");
    scanf("%lu",&N);
    F=factorial(N);
    printf("\nFactorial of %lu is : %lu",N,F);
}
```

```
    getch( );  
}
```

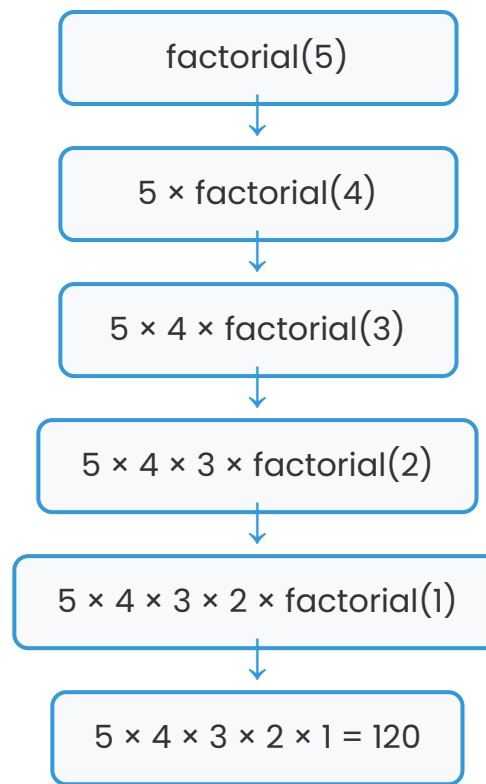
Factorial Output and Process



Output

```
Enter any Number for Calculate Factorial : 5  
Factorial of 5 is : 120
```

Process Diagram of Program:



Example 2: Display Numbers from 1 to 10



Display Numbers Using Recursive Function

```
/* display numbers from 1 to 10 using recursive function */
#include<stdio.h>
void display(int n)
{
    If (number > 10)
        return;
    else
        printf("%d\n",n);
        display(n+1);
}
void main()
{
    clrscr( );
    printf("Numbers from 1 to 10 \n");
    display(1); //1 is a starting number
    getch( );
}
```

Conclusion

In this unit, we learnt about "Functions": definition, declaration, prototypes, types, function calls datatypes and storage classes, types function invoking and lastly Recursion. All these subtopics must have given you a clear idea of how to create and call functions from other functions, how to send values through arguments, and how to return values to the called function.



Function Definition

Understanding how to define functions with proper syntax



Function Calls

Learning how to call functions and pass arguments



Recursion

Understanding self-calling functions with exit conditions



Key Points About Functions



Void Return Type

Functions that do not return any value must be declared as "void" return type



Single Return Value

A function can return only one value at a time, although it can have many return statements



Data Types

A function can return any of the data type specified in 'C'



Local Variables

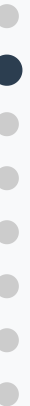
Any variable declared in functions are local to it and are created with function call and destroyed with function return



Arguments Matching

The actual and formal arguments should match in type, order and number

Recursive Termination





A recursive function should have a terminating condition i.e. function should return a value instead of a repetitive function call

